

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

Digital Radio Lab

Convolution and FIR Filtering using GNU Radio & PlutoSDR

Name	Harsh Kumar
Roll No.	226EC5003
Assigned Fc	5.003 MHz (last 4 digits of roll no.: 5003)

1. Aim

To implement Convolution and FIR Filtering in GNU Radio using a suitable test signal transmitted and received through the ADALM-PLUTO SDR, to realise convolution using a suitable impulse response (LPF / HPF / BPF / custom taps) as an FIR filter, and to display and compare the input and filtered output on appropriate GNU Radio GUI sinks. The centre frequency is set as per roll number 226EC5003 (last four digits = 5003) $\rightarrow F_c = 5.003$ MHz.

2. Software Used

- GNU Radio Companion (GRC) 3.10 – QT GUI
- Python 3 (auto-generated GRC flowgraph back-end)
- ADALM-PLUTO SDR (PlutoSDR) – used as both transmitter (Sink) and receiver (Source)
- Ubuntu Linux (host OS for GNU Radio)
- libiio / libad9361-iio drivers for PlutoSDR IIO context

3. Theory

Convolution is the fundamental operation that describes how a Linear Time-Invariant (LTI) system responds to an input signal. For a discrete input $x[n]$ and system impulse response $h[n]$, the output is:

$$y[n] = x[n] * h[n] = \sum x[k] \cdot h[n - k], \text{ summed over all } k$$

An FIR filter is a direct hardware/software realisation of this convolution sum, where $h[n]$ is a finite-length set of filter coefficients (taps). By choosing $h[n]$ appropriately — using window/frequency-sampling design routines such as `firdes.low_pass`, `firdes.high_pass`, `firdes.band_pass`, or an arbitrary custom sequence — the same convolution operation can be made to behave as a low-pass, high-pass, band-pass or custom-shaped filter. In GNU Radio, this is implemented using the Decimating FIR Filter block, which performs the convolution of the incoming sample stream with the chosen tap set in real time.

4. Source Code

This experiment was implemented entirely in GNU Radio Companion (GRC) using its graphical, drag-and-drop block-diagram environment — the Vector Source, four parallel Decimating FIR Filter blocks (LPF/HPF/BPF/custom taps), the QT GUI Chooser + Selector, PlutoSDR Sink/Source, and QT GUI sink blocks were placed and wired together directly on the GRC canvas (Section 6, Fig. 1). No manual Python/C++ coding was performed by the student. GRC internally auto-generates the equivalent Python flowgraph from this block diagram purely for execution on the back-end; since the implementation itself is block-diagram based, the auto-generated code is not reproduced here.

5. Procedure

1. Open GNU Radio Companion and create a new flowgraph; set Generate Options to QT GUI.
2. Add variable blocks: `samp_rate = 1 M`, `center_freq = 5.003 MHz` (as per roll no. 226EC5003), `tx_gain = -10 dB` and `rx_gain = 40 dB`.

- Design four impulse responses (taps) using `lpf_taps = firdes.low_pass(...)`, `hpf_taps = firdes.high_pass(...)`, `bpf_taps = firdes.band_pass(...)` and a custom_taps triangular sequence.
- Generate a test signal using a Vector Source block (repeating bit pattern) as the input to be convolved.
- Connect the Vector Source to four parallel Decimating FIR Filter blocks, each loaded with one of the four tap sets, so that each block performs convolution of the input with its own impulse response.
- Combine the four filtered outputs using a Selector block, and control which one is active using a QT GUI Chooser labelled 'Filter Select' with options LPF, HPF, BPF and custom.
- Feed the Vector Source into a PlutoSDR Sink (IIO 192.168.2.1, LO 2.4 GHz, sample rate 1 M, TX attenuation 10 dB) to transmit it, and receive it back using a PlutoSDR Source (Gain Mode: Slow Attack, RF Bandwidth 20 MHz).
- Display the received input on a QT GUI Time Sink (Rx_Time_sink) and the convolution/FIR-filtered output on a second QT GUI Time Sink (Convolution_Output_sink); display the received spectrum on a QT GUI Frequency Sink (Rx_Freq).
- Run the flowgraph and record the input and filtered-output waveforms for each of the four Filter Select settings: LPF, HPF, BPF and custom.
- Compare the time-domain and frequency-domain plots for each impulse response to verify the expected filtering behaviour of convolution.

6. Observations

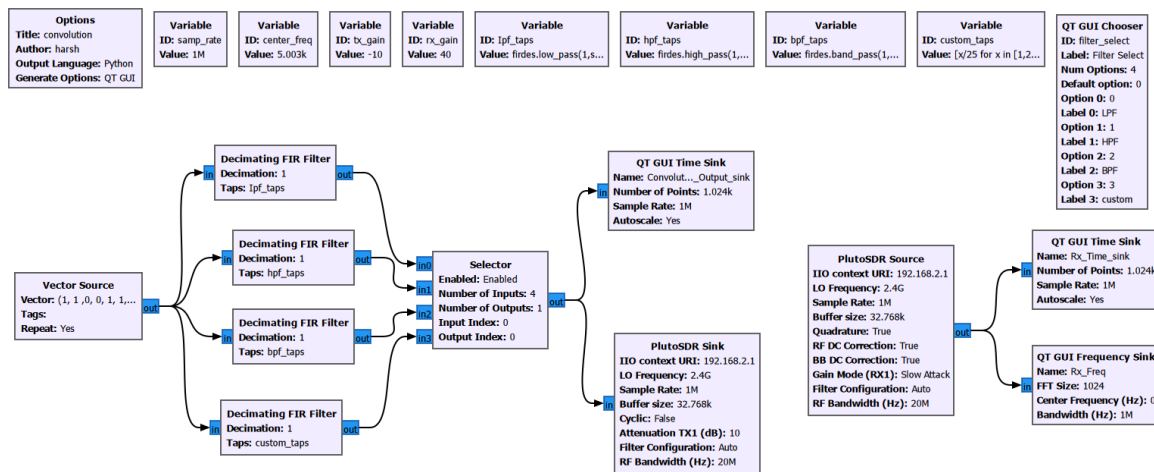


Fig. 1: GRC block diagram — Convolution / FIR filter flowgraph with four parallel Decimating FIR Filters (LPF, HPF, BPF, custom taps) selected by a QT GUI Chooser and transmitted/received via PlutoSDR.

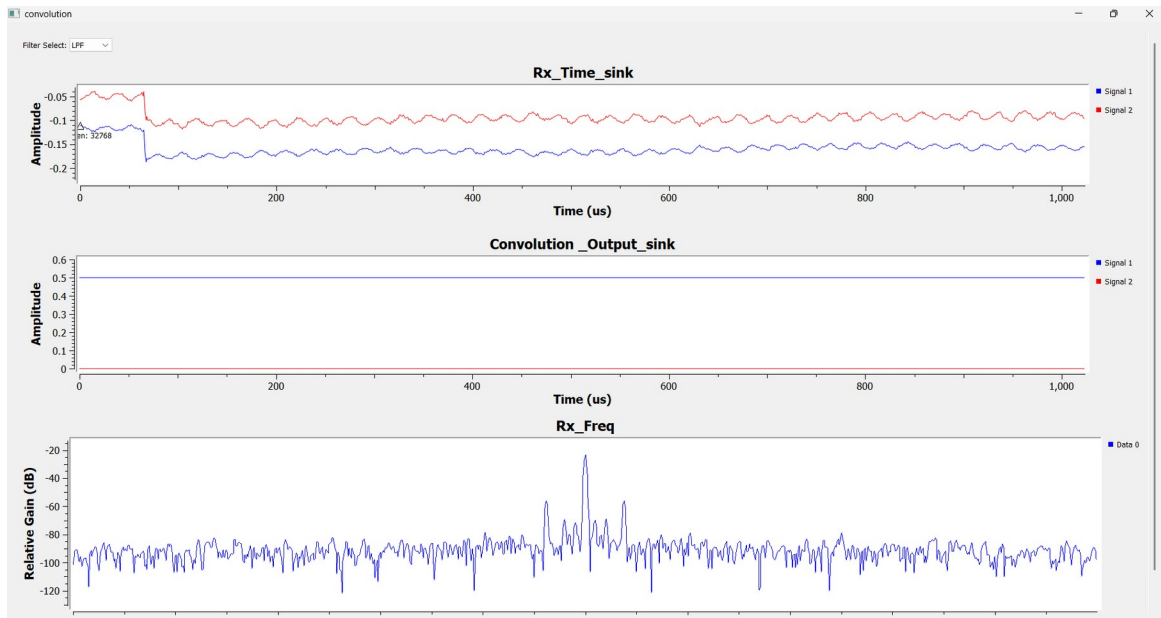


Fig. 2: Filter Select = LPF. Rx_Time_sink shows the noisy received input; Convolution_Output_sink settles to a steady, near-constant level, i.e. the impulse response has averaged out the fast variations (low-pass behaviour).

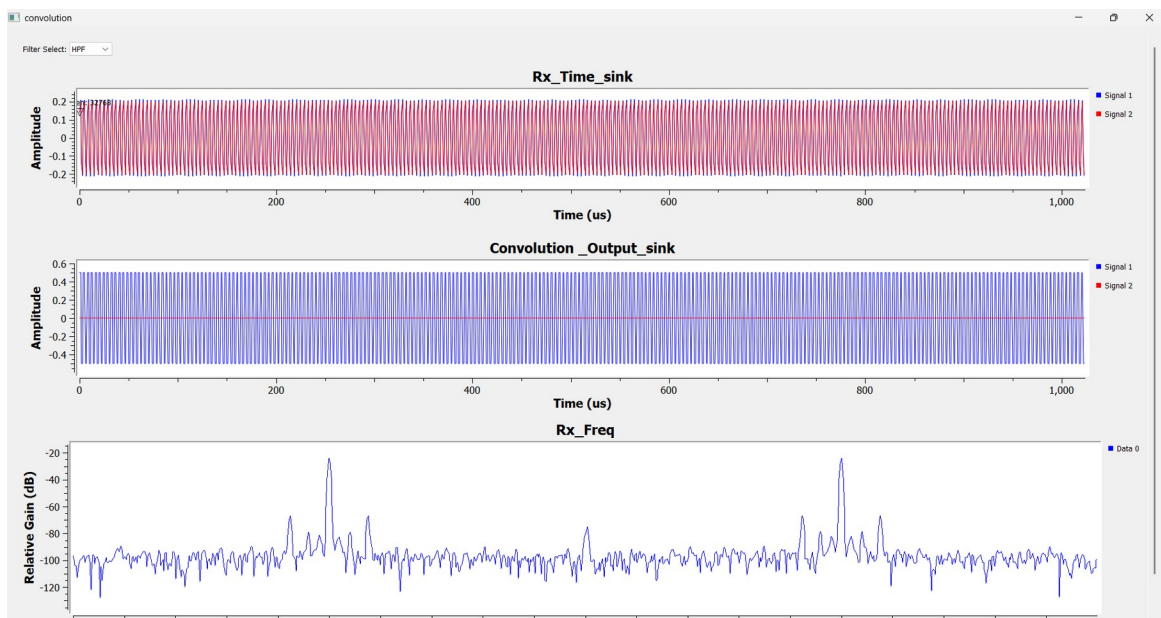


Fig. 3: Filter Select = HPF. The convolution output reproduces the fast bipolar transitions of the input while removing the DC/slow-varying component, confirming high-pass behaviour.

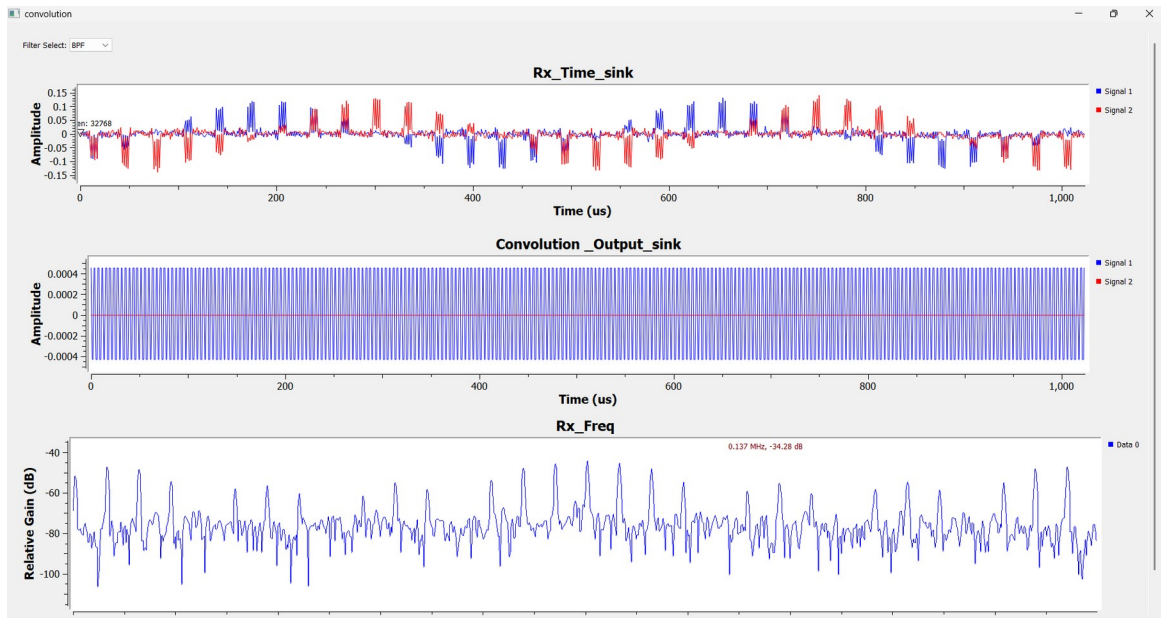


Fig. 4: Filter Select = BPF. Rx_Time_sink shows distinct amplitude bursts; the convolution output is a low-level periodic ripple, and Rx_Freq shows a marked peak at 0.137 MHz, -34.28 dB corresponding to the designed pass-band.

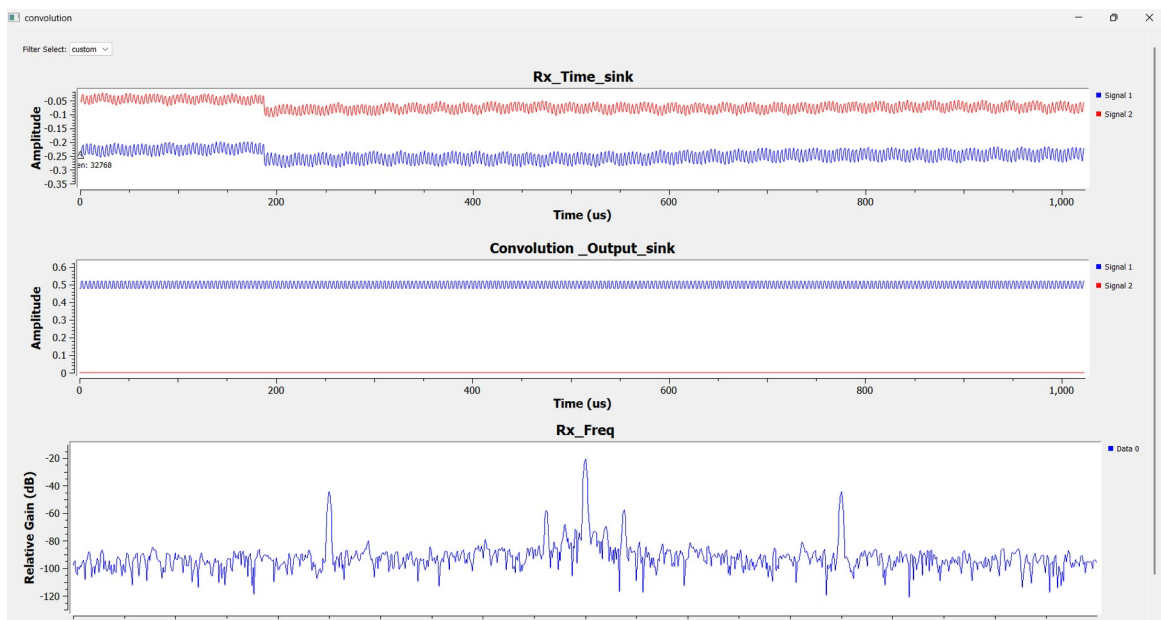


Fig. 5: Filter Select = custom (triangular impulse response). The convolution output is a smoothed, low-ripple waveform, consistent with the averaging/smoothing action of a triangular impulse response.

Impulse Response	Input (Rx_Time_sink)	Convolution Output	Rx_Freq Peak
LPF taps	Fast oscillating, noisy	Settles to near-DC level	Single peak near 0 Hz
HPF taps	Fast oscillating, noisy	Sharp bipolar spikes retained	Peaks away from 0 Hz
BPF taps	Amplitude-modulated bursts	Very low-amplitude ripple	Narrow peak ≈0.137 MHz, -34.28 dB
Custom (triangular) taps	Fast oscillating, noisy	Smoothed / averaged output	Multiple side-lobes, one main peak

7. Results & Discussion

Convolution of the received test signal with four different impulse responses (LPF, HPF, BPF and a custom triangular response) was successfully carried out in real time using the Decimating FIR Filter block, over a PlutoSDR Tx-Rx link operating with a centre-frequency offset of 5.003 MHz, 1 MS/s sample rate, TX attenuation 10 dB and RX gain 40 dB (Slow Attack).

The results confirm the expected behaviour of convolution as a filtering operation: convolving with the low-pass impulse response removed the fast variations and left a near-DC output (Fig. 2); convolving with the high-pass impulse response preserved the fast transitions while suppressing the DC/low-frequency component (Fig. 3); convolving with the band-pass impulse response retained only a narrow frequency band around 0.137 MHz while heavily attenuating everything else, resulting in a very low output amplitude (Fig. 4); and convolving with the custom triangular impulse response produced a smoothed, averaged version of the input, as expected of a moving-average-like kernel (Fig. 5).

The Rx_Freq plots corroborate the time-domain observations — the dominant spectral peak shifts/attenuates according to the pass-band of the impulse response used, verifying that the FIR filter block correctly implements the convolution sum $y[n] = x[n] * h[n]$ in real time on live SDR data.

8. Conclusion

Convolution was successfully implemented as an FIR filtering operation in GNU Radio using a PlutoSDR-based test signal at the assigned centre frequency of 5.003 MHz. By convolving the received signal with four different impulse responses (LPF, HPF, BPF and a custom taps), the corresponding low-pass, high-pass, band-pass and smoothing/averaging behaviours were clearly demonstrated and verified using QT GUI Time and Frequency Sinks. The experiment confirms that an FIR filter is, in essence, a direct hardware realisation of the discrete convolution sum, and that the shape of its output is entirely governed by the choice of impulse response (filter taps).